

DATA STRUCTURES & ALGORITHMS
SELECTION SORT

Symptom Sync Recommendation Engine



a) ALGORITHM IMPLEMENTED

SELECTION SORT

A comparison-based sorting algorithm that repeatedly finds the highest (maximum) element from the unsorted portion and swaps it to its correct position.

Time Complexity: $O(n^2)$



b) PURPOSE

In JamuSync's Symptom Sync feature, customers select the symptoms they are feeling. The system scores every jamu product based on how many of the selected symptoms it can treat.

Selection Sort orders the matches so the highest-scoring jamu appears first and receives the "BEST MATCH" badge, ensuring users get the most relevant recommendation.



WORKFLOW

- 1 Customer selects symptoms (e.g., Stomach Ache, Fatigue, Low Immunity)
- 2 Score every Jamu Product
Count how many symptoms match with each product
- 3 Store in ArrayList<Match>
Each Match contains product, score, and percentage
- 4 Selection Sort
Sort matches by highest score (first position is best match)
- 5 Display Top 3 Results
Best match shown with a "BEST MATCH" badge



THE CODE (SymptomSyncController.java)

```
// Sort so the best score is first (simple selection sort).
for (int i = 0; i < matches.size(); i++) {
    int best = i;
    for (int j = i + 1; j < matches.size(); j++) {
        if (matches.get(j).score > matches.get(best).score) {
            best = j;
        }
    }
    Match temp = matches.get(i);
    matches.set(i, matches.get(best));
    matches.set(best, temp);
}
```

Match Class

```
class Match {
    Product product;
    int score;
    int percent;
}
```

product = the jamu drink

score = number of symptoms matched

percent = score as percentage



EXAMPLE: SORTING MATCHES

BEFORE SORTING (Unordered Scores)

Temulawak



score: 1
25%

Kunyit Asam



score: 3
75%

Beras Kencur



score: 2
50%

Wedang Jahe



score: 1
25%



AFTER SORTING (Best Remedy First)

★ BEST MATCH

Kunyit Asam



score: 3
75%

2 Beras Kencur



score: 2
50%

3 Temulawak



score: 1
25%

Wedang Jahe



score: 1
25%



c) CLASSES IN APPLICATION CONTEXT

Class: SymptomSyncController
Handles symptom selection, scoring, sorting, and displaying results.

- **rebuildResults()**
Scores all products and sorts matches using Selection Sort.
- **matchesAnyKeyword()**
Checks if a product's description, ingredients, or health benefits contain a symptom keyword.
- **createProductCard()**
Creates the result card. Index 0 gets the "BEST MATCH" badge.

Helper Class: Match

Attributes

- Product product
- int score
- int percent

Represents one match between a product and the user's selected symptoms.



SYMPTOM SYNC SCREEN (Actual UI)

Selected Symptoms (3)

Stomach Ache X
Low Immunity X
Fatigue X

Add Symptoms

Headache Cough Bloating
Stress Cold Nausea

Top Matches

- ★ BEST MATCH

Kunyit Asam
Helps with stomach health, boosts immunity, reduces fatigue and maintains body stamina. **75% match**
- Beras Kencur**
Helps with digestion, reduces fatigue, and improves appetite. **50% match**
- Temulawak**
Supports liver health, improves digestion, and boosts overall immunity. **25% match**

Showing top 3 recommendations

WHY SELECTION SORT?

- ✓ Simple and easy to understand and implement
- ✓ Easy to reason about and debug
- ✓ Our match list is small (only products that scored above zero) — so $O(n^2)$ is perfectly acceptable here.



Built with



JavaFX



TEAM QUANTALABS

Hassan Waleed Hassan Abdo Al-Awadhi	25523274
Laraib Arshad	25523275
Nada Taufik Shahbal	25523266
Rama Walta Alinta Elsaprike	25523085

GitHub Repository

